

SIGN LANGUAGE CONVERTER

By

BSE24-14

MACHINE LEARNING MOBILE APPLICATION

DEPARTMENT OF NETWORKS

SCHOOL OF COMPUTING AND INFORMATICS TECHNOLOGY

A Project Report Submitted to the School of Computing and Informatics Technology

for the Study Leading to a Project in Partial Fulfillment of the

Requirements for the Award of the Degree of Bachelor of

Science in Software Engineering of Makerere University.

Supervisor

Dr. Bernard Muwonge Ssajjabi

Department of Networks

School of Computing and Informatics Technology, Makerere University

bmuwonge@cit.ac.ug, +256-41-540628, Fax: +256-41-540620 {note: numbers are standard

May, 2024

Declaration

We, group BSE 24-14, hereby declare that the work presented is original and has never been submitted for an award to any university or institution of higher learning. We can confirm that where we have done consultations either from published material or the works of others, it has been attributed in this report.

#	Names	Registration Number	Signature
1	Suubi Trevor John	20/U/2045/EVE	
2	Nantanda Jamilah	20/U/7807/PS	
3	Mulungi Steven Junior	20/U/23507/PS	
4	Nabwire Esther	20/U/2014/EVE	

Approval

This project report titled "SIGN LANGUAGE CONVERTER" has been submitted for examination with my approval as the supervisor of group BSE 24-14

Dr. Bernard Muwonge Ssajjabi

Department of Networks

School of Computing and Informatics Technology;

College of Computing and Information Sciences,

Makerere University

Signature: Date:

Supervisor

Dedication

Acknowledgments

Abstract

Table of Contents

Declaration.....	2
Approval.....	3
Dedication.....	4
Acknowledgments.....	5
Abstract.....	6
Table of Contents.....	7
List of Figures.....	9
List of Tables.....	10
Abbreviations/Acronym.....	11
Chapter 1: Introduction.....	13
1.1 Background and scope of the project.....	13
1.2 Overview of the document.....	13
Chapter 2: System Specifications.....	13
2.1 Version of requirements and Version Control.....	13
2.2 Input.....	13
2.3 Output.....	14
2.4 Functionality.....	14
2.5 Limitations and Safety.....	14
2.6 Default settings.....	15
2.7 Special requirements.....	15
2.8 Errors and alarms.....	15
Chapter 3: Design output.....	16
3.1 Implementation (coding and compilation).....	16
3.2 Documentation.....	16
Chapter 4: Inspection and testing.....	18
4.1 Introduction.....	18
4.2 Test plan and performance.....	19
4.2.1 Test objectives.....	19
4.2.2 Scope and Relevancy of Tests.....	19
4.2.3 Levels of tests.....	19
4.2.4 Types of tests.....	19
4.2.5 Sequence of tests.....	19
4.2.6 Configuration and calculation tests.....	19
4.3 Precautions.....	20
4.3.1 Anomalous conditions.....	20
4.3.2 Precautionary steps taken.....	20

Chapter 5: Installation and system acceptance test.....	21
5.1 Input files.....	21
5.2 Supplementary files.....	21
5.3 Installation qualification.....	21
Chapter 6: Performance, servicing, maintenance, and phase out.....	23
6.1 Service and maintenance.....	23
6.2 Performance and Maintenance.....	23
Chapter 7: Conclusion and Recommendations.....	26

List of Figures

List of Tables

Abbreviations/Acronym

Table 1.1 shows the acronyms and their meanings

Acronym/Abbreviation	Meaning
DHH	Deaf and Hard of Hearing
UI	User Interface
TTS	Text-to-Speech
API	Application Programming Interface
SRS	Software Requirements Specification
SDD	Software Design Document
SLC	Sign Language Converter
iOS	iPhone Operating System
IDE	Integrated Development Environment
HTTP	Hyper Text Transfer Protocols

Chapter 1: Introduction

1.1 Background and scope of the project

Human beings interact with each other to convey their ideas, thoughts, and experiences to the people around them. But this is not the case for deaf-mute people. Sign language paves the way for deaf-mute people to communicate. [1]

Sign language is a vital means of communication for the deaf/mute community. However, a gap in communication arises when sign language is not widely taught in educational institutions and workplaces, making it challenging for those who do not understand it to communicate effectively with the deaf/mute population.

But not everyone knows sign language, it takes time and is usually expensive to learn. To allow sign language practitioners and non-sign language practitioners to communicate effectively and affordably the Sign Language Converter (SLC) was proposed.

The SLC is a mobile application that will be designed to convert sign language gestures into text or speech. The application was developed for Android platforms and targeted users within the Deaf and Hard of Hearing (DHH) community, as well as individuals who wish to communicate with sign language users.

The application has the following objectives:

- To provide real-time translation capabilities, allowing users to communicate effortlessly across spoken and signed languages.
- To allow sign language users to communicate with non-sign language users through recognizing and converting the signer's hand gestures into text or speech.
- To allow users to create and edit profiles, for a custom user experience.
- To include features that support sign language learners by providing feedback on correct signing techniques and offering educational resources.

Benefits of using the application will include:

- Facilitates communication between members of the DHH community and those who may not be proficient in sign language, fostering understanding and inclusivity.
- Empowers individuals with hearing impairments by providing them with a tool to express themselves effectively in various settings.
- Supports the education of sign language, serving as a valuable resource for learners and educators.
- Affordable, since there's no need to pay interpreters to do the translation. Page 1 Software Design Document
- Promotes inclusivity in various fields such as education, healthcare, etc. That is, it makes it easier for the DHH community to be involved in various fields of industry.

1.2 Overview of the document

This document describes the implementation, testing, and validation findings for the Sign Language Converter system. It is divided into the following sections:

Chapter 1: Introduces the project, explaining its importance and objectives, particularly focusing on the need for a sign language conversion mobile application, and provides a summary of the document's structure, outlining the contents and purpose of each chapter. This section gives an overview of the document

Chapter 2: Details the system's technical specifications, including version control, inputs, outputs, functionalities, limitations, safety precautions, default settings, special requirements (e.g., security, data backup), and error handling. It provides a foundation for validating the system's capabilities and understanding its constraints.

Chapter 3: Discusses development tools, coding practices, integration details, device interfaces, and operating environments used in system implementation, and describes the types of documentation provided and their roles for users and developers.

Chapter 4: Outlines the inspection and testing processes for the sign language conversion app. It covers the test plan, including objectives, scope, levels, types, sequence, and configuration tests.

The chapter emphasizes ensuring compliance with requirements and managing anomalous conditions to guarantee system reliability and stability

Chapter 5: Covers the proper installation and validation of the system. It lists input and supplementary files, provides steps to verify correct component installation, and includes a checklist to document and approve the installation process, ensuring the system functions correctly before deployment.

Chapter 6: Details the performance, servicing, and maintenance requirements, outlining support processes, maintenance schedules, updates, problem solutions, and performance criteria. It includes strategies for system upgrades, functional expansions, and transitioning from the old system to the new one, ensuring smooth data migration and revalidation.

Chapter 7: Summarizes the entire report, highlighting key findings, conclusions, and recommendations for future improvements.

Chapter 2: System Specifications

The section describes and specifies the system completely and is the basis for the validation process.

2.1 Version of Requirements and Version Control

Version of, and changes applied to, the requirements specification. Specify the version of your requirements document. Remember you used to make changes. If you made changes between versions, please specify which changes were made to get which versions. Also, explain why the changes were made

How to identify different versions of the computer system and to distinguish output from the individual versions. If you have different versions of your software, how do you know which version is 1.0, or 1.2?? Did you use tools like GIT, TOTROISE ETC to manage versioning of your code?

The Software Requirements Specification version is 1.0, the first and final draft where the requirements were stated as perceived from data collected from the sign language users, non-sign language users, and literature. Only minor changes were made to this version, these include grammatical error correction and other document structure improvements. Other changes include:

- Editing of the requirements to include how the requirement will be achieved.
- Removal of a Safety Requirement, that allowed users to terminate the application in case of a bug or failure.

The Sign Language Converter application has only one version at the moment, 1.0.0, and the version control system that was used to manage updates and changes to the application was Github. A GitHub repository was used to maintain a history of changes to the application and facilitate collaboration among the development team.

The version control process involved creating branches for different features or improvements. Once a commit to the repository passed specific checks it would be merged with the main or master branch.

2.2 Input

All inputs the computer system will receive. Include ranges, limits, defaults, responses to illegal inputs, etc. Please explain in detail what your inputs are for example, if one has a security system GPS coordinates:

Input 1: The coordinates are received from the GPS system. When one is attached, the system picks the coordinates and extracts the corresponding location name.... which is sent to all phone numbers in the contact list.

Input 2: (please don't simply state the inputs, go ahead and explain them so that the reader fully understands your system)

This subsection details all inputs the application will receive and includes ranges, limits, defaults, and responses to illegal inputs. The inputs are listed below as follows:

Input 1: Account information. This is entered by a user on signing up or creating an account, this includes a name, email, password, and phone number. The name should be 2 words, with a space in between, the email should be valid, the password should be greater than 8 characters, contain at least one lowercase and uppercase letter, 1 digit, and one special character, and the phone number should be exactly equal to 10 digits.

Input 2: Camera feed. This includes the information that is picked by the smartphone camera and processed by the computer vision algorithm to convert the gestures into text. This information includes the frames and bytes of the image.

Input 3: Quiz answers. This information is entered by the user as they are attempting questions in the sign language quiz. Each question requires a one-letter answer which can be either uppercase or lowercase.

2.3 Output

All outputs the computer system will produce. Includes data formats(eg. Images, text, video, etc.), screen presentations, data storage media, printouts, automated generation of documents, etc. Do a thorough explanation of all the outputs here

Output 1: Converted text. This is the text that is displayed to the user after the recognition of a sign gesture.

Output 2: Speech output. The text displayed on the screen can be converted to speech as well with a press of a button by leveraging the flutter_TTS package.

Output 3: Quiz verification remarks. These are comments that are displayed below the submit button while the user attempts the sign language quiz. For each answer the user submits, a comment is displayed verifying whether the answer is correct or not.

Output 4: Quiz score. A final score is displayed at the end of the quiz, along with a comment. This score grades how well the user attempted the quiz.

2.4 Functionality

All functions the computer system will provide. Includes performance requirements, such as data throughput, reliability, timing, user interface features, etc. Explain all the functionalities of your system in detail.

Sign language gesture capture and recognition: The application employs a computer vision algorithm that uses a YoloV8 object detection model trained with a curated sign language dataset to accurately detect, analyze, and interpret the various gestures and signs used in sign language by the user.

Text-to-Speech (TTS) conversion: After the sign gestures are translated, they are transformed into audible speech with the help of the flutter_TTS package in Flutter which uses native Text-to-Speech capabilities to mimic the sound and tone of natural human speech. The resulting output allows individuals who are deaf or hard of hearing to communicate more effectively with those who do not understand sign language.

Learning Module: The application is equipped with a range of features that are particularly helpful for those who are learning sign language. The software offers a range of educational resources that are designed to help users learn and master the basics of sign language, making it an indispensable tool for anyone serious about learning this valuable skill.

Profile customization: The system was designed to offer users the flexibility to personalize and modify their profiles according to their specific requirements and preferences. With this feature, users can add, edit, or remove information, including their account information, and other interests like their hearing status and language preferences. Additionally, the system provides users with a user-friendly interface that makes customization and editing of profiles easy.

2.5 Limitations and Safety

*All acceptable and stated limitations in the computer system. What limits the use of the system? Please identify the limitations of the system **during its use** and NOT during development. Things like “The system requires an active Internet connection. If there is no Internet connection, messages sent from users cannot be received.” Etc. For embedded systems, they have energy/power limitations, web systems have ...*

All precautions are taken to prevent overflow and malfunction due to incorrect input or use. When the system is in use, such and such must be done to avoid incorrect input or use of the system. E.g. “The administrators shall enter all eligible districts. This is to ensure that only eligible villages are entered into the system.” (Give others)

i) Lighting conditions: Adequate lighting conditions are necessary for the application to perform the sign gesture recognition more accurately.

ii) Resource usage: Running the model on a smartphone takes time because the model is large and requires more processing power. So users may be required to clear application cache, and stop heavy running tasks in the background or foreground for the translation to take place smoothly. The translation may also lead to the heating up of the phone due to the camera running.

iii) Basic sign language knowledge: The learning module resources that are provided cover basic sign language concepts including the alphabet and numbers from 0 to 10.

2.6 Default settings

All settings applied after power-up such as default input values, default instrument or program control settings, and options selected by default. Includes information on how to manage and maintain the default settings. EG By default, when the system is first installed, it contains only one user (give the username and password). For security reasons, the password can be changed via (give the menu name). Etc.

i) User email and password updates: For security reasons, users are not allowed to update their emails and passwords.

ii) TTS voice settings: The default voice used by the TTS service depends on the device's TTS settings, a device using Google TTS as the default TTS engine, will use the default voice configured for the selected language which is English.

iii) Connectivity: The first-time usage of the application requires an active and healthy internet connection.

iv) Hearing and Language Preferences: By default, the preferences for hearing status and language are set to “Hearing” and “English” respectively. These preferences do not affect the user experience currently but may be used in future developments or enhancements to offer a custom user experience.

2.7 Special requirements

Requirements the laboratory is committed to, security, confidentiality, change control and back-up of records, protection of code and data, precautions, risks in case of errors in the computer system, etc.

The application uses Firebase services that is, Firebase Authentication and Firestore to authenticate users and store user data securely since Firebase Authentication employs secure practices, such as using industry-standard OAuth and OpenID Connect protocols.

The application can work offline by leveraging Firestore’s offline persistence which involves Firestore caching data locally on the device such that it can be accessed even when there’s no internet.

The application processes and translates sign language gestures in real-time, it’s able to accomplish this through the object detection model that runs on the application using the device’s camera, because of this it needs at least 3GB of memory, and the device may heat up during translation.

2.8 Errors and alarms

Identify the errors that could arise from the use of the system. How would you handle the errors?

i) Inaccurate translations: This may be due to insufficient lighting, background interference (noise), or the model’s inefficiencies which can be solved by training the model on a wider dataset, and other machine learning techniques such as increasing the number of epochs.

Chapter 3: Design output

3.1 Implementation (coding and compilation)

Development tools used to implement the system, notes on anomalies, module and integration details, etc.

All device interfaces and equipment to be supported. Identify the devices and explain how they are used in your system. For those doing embedded systems, you have sensors, key pads etc., mobile applications, you have mobile phones, web, you have computers ...

The hardware and software operating environment in which to use the computer system. E.g. laboratory or office computer, the actual operating system, network, third-party executables such as Microsoft Excel and Word, the actual version of the platform, etc.

This subsection details the implementation of the SLC application, the programming languages, and the technologies used as well as the environment in which the application operates.

Development Tools

i) Programming languages and other technologies used: The application was written and developed using the Flutter framework with Dart as the primary programming language. Firebase was employed for backend support, database management, and user authentication. The object detection model was developed in Python with other services such as Roboflow being used for labeling, classifying, and augmentation of the images for the sign language dataset and Google Collab for training the model.

ii) Visual Studio Code: The application was developed using Visual Studio Code (VS Code) a lightweight, versatile code editor with a Flutter extension that provides Flutter-specific functionalities, including hot reload, debugging, and Dart code analysis.

iii) Android smartphone: The application was run and tested on phones that have an Android operating system and support USB debugging.

Module and Integration details

The application was split into modules, to help make the architecture more modular and less monolithic, ensuring that the components of the application are loosely coupled and make the codebase easier to maintain and test.

- i) **Gesture Capture and Recognition:** This module is responsible for capturing the hand gestures the sign language users make, processing them, and converting them to text using computer vision algorithms. It then passes this text to the TTS module.
- ii) **Text-to-Speech Module:** The module is responsible for converting text displayed on the screen into speech for the user using a synthetic voice. It uses the text converted by the Gesture Capture and Recognition module.
- iii) **Learning Module:** This section of the application contains a sign language dictionary showing all the letters of the English alphabet and numbers, their respective sign gestures, as well as quizzes to challenge non-sign language users and help them learn.
- iv) **User and Profile Management:** This module allows users to create and manage user profiles within the application.

Operating Environment

The application is designed for devices running the Android Operating System (OS) and targets versions of Android 7.0 (Nougat) and above ensuring a wide user base. It requires access to the device's main camera.

3.2 Documentation

What Kind of Documentation has been provided as output from the Design and what is its role to the readers?

The Software Design Document (SDD) was developed as output from the design process and it describes the architectural and system design of the Sign Language Converter (SLC) project. It provides a detailed overview of the system components, their interactions, and the design

decisions made during the development process. The primary audience for this document included the software developers, project supervisors, and other stakeholders involved in the Sign Language Converter project.

Table No: Design details (check all that apply to your project. Make sure you can defend what you tick)

<i>Topics</i>	Design output	
Good programming practice <i>Efforts made to meet the recommendations for good programming practice...</i>	Source code is... ☒ Modularized ☒ Encapsulated ☒ Functionally divided ☐ Strictly compiled ☒ Fail-safe (handling errors)	Source code contains... ☐ Revision notes ☒ Comments ☒ Meaningful names ☒ Readable source code ☐ Printable source code
Dynamic testing <i>Step-by-step testing made dynamically during the implementation...</i>	☒ All statements have been executed at least once ☒ All functions have been executed at least once ☐ All case segments have been executed at least once ☒ All loops have been executed to their boundaries ☐ Some parts were not subject to the dynamic test Comments:	

Chapter 4: Inspection and testing

4.1 Introduction

Software testing is the process of evaluating a software product to find out whether it's working as intended, while software inspection is additional testing done in conjunction with the code review to ensure that the code is correct, optimized, and maintainable.

Table No: Inspection plan and performance

<i>Topics</i>	3.3.1 Inspection plan and performance	<i>Date / Initials</i>
Design output <i>Results from the Design Output section inspected...</i>	☒ Program coding structure and source code ☒ Evidence of good programming practice ☒ Design verification and documented reviews ☐ Change-control reviews and reports Comments:	20th/05/2024 STJ, JN, NE, MS
Documentation <i>Documentation inspected...</i>	☐ System documentation, flow charts, etc. ☒ Test results ☒ User manuals, On-line help, Notes, etc. ☐ Contents of user manuals approved	26th/05/2024 MS, JN, NE, STJ

	Comments:	
Software development environment <i>Environment elements inspected...</i>	☒ Data integrity ☐ File storage ☐ Access rights ☐ Code protection ☐ Installation kit, replication, and distribution Comments:	28th/05/2024 JN,MS,NE,STJ
Result of inspection <i>Approval of inspection.</i>	☒ Inspection approved Comments:	30th/05/2024 NE,JN,MS,STJ

4.2 Test plan and performance

The central activity in Sign Talk Application test design was to identify possible sets of inputs and the expected outcomes from the system to verify whether the system possesses the set of related requirements as specified in its Software Requirements Specification (SRS) document.

Item to test	How to test	Expected Output	Approved

4.2.1 Test objectives

Test objectives were identified from the requirements specification from which Sign Talk Application test objectives and test case designs were deduced. The main objective was to verify that the functionality of Sign Talk Application works according to its SRS. The other objectives were as follows;

Table: Test description.

What was tested	How it was tested	Why it was tested

To verify the user account creation and login process

This test confirms that a user can successfully register an account and login into it without risking data loss or any security vulnerabilities.

The test involved creating accounts using valid and invalid input data, verifying data storage in the database and checking for successful login using the registered credentials, all on an Android smartphone.

To validate the text or speech output in regard to the sign gestures captured

This test confirms the accuracy of the SLC by checking the correctness of text or speech output against the sign language gestures that are recorded.

Different sign gestures were recorded and the following aspects were observed; correctness of text output, clarity of speech output, duration of translating the sign gesture and number of times errors were returned.

To determine the usability of the Learn feature.

This test determines how smooth a user's educational experience on the SLC is by efficiently accessing the learning material provided and attempting the quiz in the Learn feature.

The Learn feature of the SLC was accessed with the application running on an Android smartphone. The test involved trying to access the learning material provided, attempting the quiz and verifying its grading functionality, exiting the quiz without full attempt.

To determine the ease of profile customization.

This test helps us to understand the SLC's flexibility, as well as a user's experience as they customize their profile.

An already created account was accessed and the different profile customization options were tested. This includes modifying profile picture.

4.2.2 Scope and Relevancy of Tests

The tests included all the functionalities of the system. They were done on individual components to avoid complexity after integration. The scope of the tests conducted were based on code coverage and module-based coverage. We identified components of the system to be tested which we referred to as "in scope " and those that are not to be tested as "out of scope". The tests were designed to cater for validation and verification testing of Sign Talk Application. Verification covered the application requirements identified during the requirements phase. Validation testing was employed at the implementation phase against the documented WWC design document. Testing ensured the following:

- It helped to inspire customer confidence and satisfaction where it provided visibility on how well and stable the system has been developed.
- It helped to reduce risks where defects were sought out and fixed or removed.
- It helped to ensure high quality of the product where quality was measured by the number of defects identified during testing.
- It helped the project manager have a better control over the budget.

Table:Test Coverage and Objective met.

System Requirements	Test Coverage (%)	Test Objective met

4.2.3 Levels of tests

Module test, integration test, and system acceptance test.

Module test:

The SLC comprises two major modules, that is, the Translation module and the Learning module. Each module was tested in isolation to ensure that it functions properly. The other components that were tested include

Integration test:

System acceptance test:

4.2.4 Types of tests

E.g. input, functionality, boundaries, performance, and usability.

Basic tests:

These tests provide evidence that the SLC can be installed, configured, and brought to an operational state on an Android smartphone of the required specifications..

Functionality tests:

These tests provide a comprehensive coverage of the account creation, login, gesture translation, profile customization and learning processes of the SLC in order to ensure they meet the expected requirements.

Performance tests:

These tests measure the response time of the translator feature, as well as, how well the SLC works under varying lighting conditions.

Input tests:

These tests are useful in validating and categorizing data input as valid or invalid for efficient creation of user accounts and logging in.

Usability tests:

These tests evaluate a user's experience with the SLC, in terms of how easy it is to navigate, its complexity and responsiveness.

4.2.5 Sequence of tests

Test cases, test procedures, test data, and expected results.

4.2.6 Configuration and Calculation Tests

Platform, network, and integration with other systems.

Calculation tests confirm that known inputs lead to specified outputs.

4.3 Precautions

4.3.1 Anomalous conditions

When operating in a third-party software environment, such as Microsoft[®] Windows, and Office, some undesirable, inappropriate, or anomalous operating conditions may exist. A discrepancy

between the description of the way an instrument should operate, and the way it actually does, may be regarded as an anomaly as well. Minor errors in a computer system may sometimes be acceptable if they are documented and/or properly circumvented. Anomalous operating conditions associated with the computer system itself, Anomalous operating conditions in the used instruments.

4.3.2 Precautionary steps taken

The steps taken to workaround anomalous, inappropriate, or undesired operating conditions are verified and tested. Precautionary steps taken in e.g. Windows settings, Precautionary steps taken to workaround problems with the used instruments, Precautionary steps taken to workaround problems with the computer system itself.

Chapter 5: Installation and system acceptance test

The validation of the installation process ensures that all system elements are properly installed in the host system and that the user obtains a safe and complete installation, especially when installing software products.

5.1 Input files

List of (relevant) files on the installation media and what they are used for

5.2 Supplementary files

Readme files, License agreements, examples, etc.

5.3 Installation qualification

Steps to ensure and document that each component is installed correctly.

Table no: Checklist of the Installation and system acceptance test

<i>Topics</i>	Installation summary
Installation method <i>Automatic or manual installation...</i>	☒ Automatic - installation kit located on the installation media ☐ Manual - Copy & Paste from the installation media Comments:
Installation media	☐ Diskette(s) ☐ CD-ROM ☐ Source disk folder (PC or network)

<i>Media containing the installation files...</i>	☒ Download from the Internet Comments:
Installed files <i>List of (relevant) installed files, e.g. EXE- and DLL-files, spreadsheet Add-ins and Templates, On-line Help, etc.</i>	• APK files

Table no: Installation Procedure Check

<i>Topics</i>	Installation procedure	<i>Date / Initials</i>
Authorization <i>Approval of installation in actual environment.</i>	Person responsible:	
Installation test <i>The following installations have been performed and approved...</i>	☐ Tested and approved in a test environment ☐ Tested and approved in actual environment	

	☐ Completely tested according to test plan ☐ Partly tested (known extent of update) Comments:	
--	--	--

Chapter 6: Performance, servicing, maintenance, and phase out

6.1 Service and maintenance

Documentation of service and support concerning maintenance, future updates, problem solutions, requested modifications, etc.

6.2 Performance and Maintenance

Which are the requirements for service, maintenance, performance (Maximum time taken before giving output) , and support(what kind of support do you expect to give to the clients at this stage). This phase is where all activities during performance reside and where decisions about changes (What would be the causes of incorporating changes?), upgrades(How should the software be upgraded?), revalidation, and phase out are made(How do you move from the old system to the new one you have just come up with?? Ie How do you move data from the old system to the new system?). [EXPLAIN THESE IN SOME PARAGRAPHS /PARAGRAPH AND LATER FILL THE TABLE BELOW]

Table no: Performance and maintenance details

<i>Topics</i>	Performance and maintenance	<i>Date / Initials</i>
Problem / solution	<i>Detection of system problems causing operating troubles. A first step could be to suggest or set up a well-documented temporary solution or workaround.</i>	Dates must be filled in

Functional maintenance	<i>E.g. if the computer system is committed to international standards, and these standards are changed, the computer system, or the way it is used, should be updated accordingly.</i>	
Functional expansion and performance improvement	<i>List of suggestions and requests, which can improve the performance of the computer system. eg</i> · Inclusion of a PH sensor to determine the soil PH levels to allow for automatic neutralization of the soil. · Use of wireless technology instead of wired connections.	

Chapter 7: Conclusion and Recommendations

Make a conclusion of your whole report

Appendix A: User Manual

Give details on how the system can be used, where to go for help etc accompany your explanations with screenshots

(For those doing embedded systems, please add your assembled diagram, those doing other projects can add some screenshots to their manual. The screenshots must appear in a logical order)

Final approval for use	
Identification:	
Responsible for validation:	
Remarks:	
Date:	Signature:

References

1. M. A. P. Vijayalakshmi, "Sign language to speech conversion," *International Conference on Recent Trends in Information Technology (ICRTIT)*, pp. 1-6, 2016.